L39: Agentic Concolic Execution

LLM agents as the symbolic engine of a concolic tester

Software Analysis

IIT Guwahati · Nov 5, 2026

Agentic Concolic Execution

Zhengxiong Luo*, Huan Zhao*, Dylan Wolff*, Cristian Cadar†, Abhik Roychoudhury*
* *National University of Singapore*
† *Imperial College London*
{luozx, abhik}@nus.edu.sg, {zhaohuan, wolffd}@comp.nus.edu.sg, c.cadar@imperial.ac.uk

***Abstract*—Concolic execution is a practical test generation technique that explores execution paths by coupling concrete execution with symbolic reasoning. It runs programs on given inputs while capturing symbolic path representations, then mutates and solves these constraints to generate new test inputs for alternative paths. This approach has several fundamental challenges, such as (C1) the inherent complexity of symbolically modeling diverse programming language constructs and environmental interactions, and (C2) the scalability issues of constraint solvers when handling large, complex formulas.**

In this work, we investigate whether LLM agents can help address these longstanding challenges in test generation. We propose a novel workflow which we call *agentic concolic execution*. Using an LLM agent for symbolization, our approach is language-agnostic and can handle environmental constraints without additional manual modeling effort. To ease pressure on

gather symbolic constraints representing the executed path, then negates selected constraints and solves for new inputs to explore alternative paths [2]. These approaches have achieved success in discovering vulnerabilities in real-world software [3], played key roles in competitions like DARPA Cyber Grand Challenges [4], [5], and proven effective in both FLOSS and commercial software [2], [6], [7].

Despite their success, these approaches still face two fundamental challenges when applied to modern software:

***C1**: Complex and Incomplete Implementations.* Modern DSE engines often integrate a symbolic interpreter with a virtual-machine-style architecture [3]. This approach enables them to collect symbolic constraints arising from diverse programming language constructs and environmental interactions, such as file and network operations. However, developing comprehensive symbolic modeling rules for all

IEEE Symposium on Security and Privacy (S&P) 2026

Concolic Execution (1/5)

```
1  int foo(int v) {
2    return 2*v;
3  }

5  void test_me(int x, int y) {
6    int z = foo(y);
7    if (z == x)
8      if (x > y+10)
9        ERROR;
10 }
```

1

run on a concrete input *I*

x = 22, y = 7, z = 14

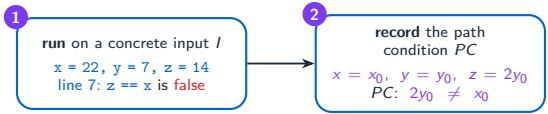
line 7: z == x is false

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026; Naik, CIS 5470, Penn: Dynamic Symbolic Execution

Concolic Execution (2/5)

```
1  int foo(int v) {
2    return 2*v;
3  }

5  void test_me(int x, int y) {
6    int z = foo(y);
7    if (z == x)
8      if (x > y+10)
9        ERROR;
10 }
```

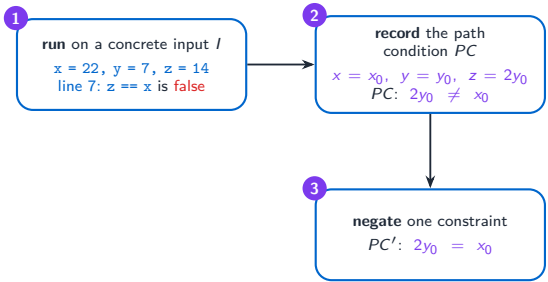


Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026; Naik, CIS 5470, Penn: Dynamic Symbolic Execution

Concolic Execution (3/5)

```
1  int foo(int v) {
2    return 2*v;
3  }

5  void test_me(int x, int y) {
6    int z = foo(y);
7    if (z == x)
8      if (x > y+10)
9        ERROR;
10 }
```

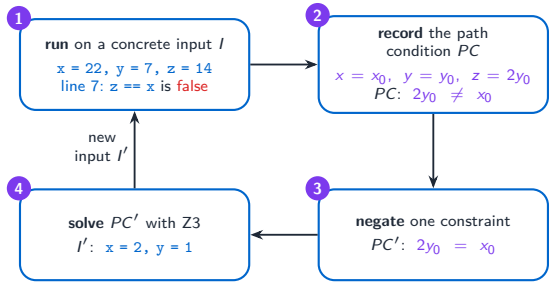


Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026; Naik, CIS 5470, Penn: Dynamic Symbolic Execution

Concolic Execution (4/5)

```
1  int foo(int v) {
2    return 2*v;
3  }

5  void test_me(int x, int y) {
6    int z = foo(y);
7    if (z == x)
8      if (x > y+10)
9        ERROR;
10 }
```

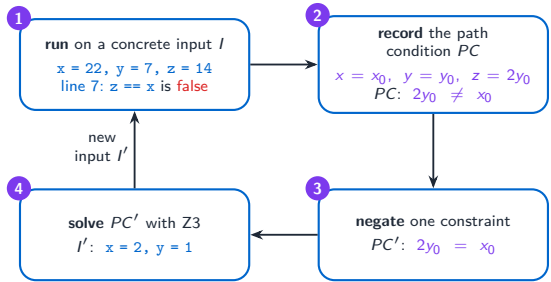


Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026; Naik, CIS 5470, Penn: Dynamic Symbolic Execution

Concolic Execution (5/5)

```
1  int foo(int v) {
2    return 2*v;
3  }

5  void test_me(int x, int y) {
6    int z = foo(y);
7    if (z == x)
8      if (x > y+10)
9        ERROR;
10 }
```

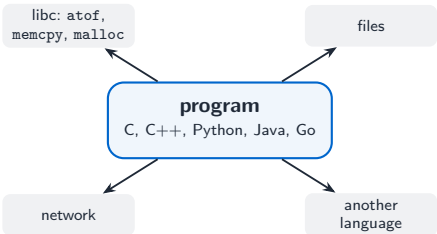


run 2: `z == x` is **true**, line 8 runs: a **new path**

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026; Naik, CIS 5470, Penn: Dynamic Symbolic Execution

Two Challenges (1/5)

Complex and incomplete implementations

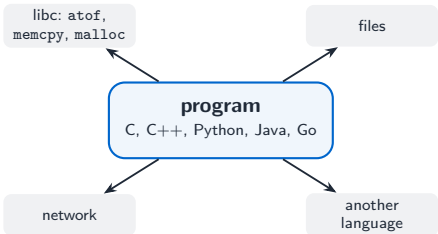


every language feature and every environment call needs a **hand-written model**

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Challenges (2/5)

Complex and incomplete implementations



every language feature and every environment call needs a **hand-written model**

Expensive constraint solving

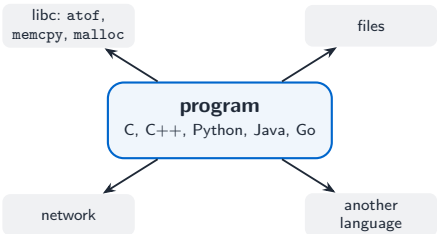
```
float x, y;
while (x != y)
  x = next(x);
```

floats in a loop

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Challenges (3/5)

Complex and incomplete implementations



every language feature and every environment call needs a **hand-written model**

Expensive constraint solving

```
float x, y;
while (x != y)
  x = next(x);
```

floats in a loop

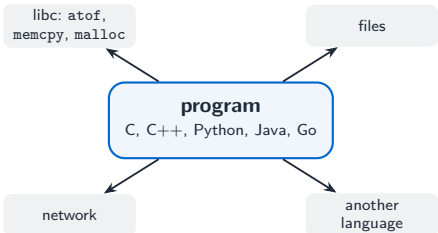
↓ each iteration adds one clause

$PC: \; x_1 \neq y \wedge \cdots \wedge x_n \neq y$

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Challenges (4/5)

Complex and incomplete implementations



every language feature and every environment call needs a **hand-written model**

Expensive constraint solving

```
float x, y;
while (x != y)
  x = next(x);
```

floats in a loop

each iteration adds one clause

$PC: \; x_1 \neq y \wedge \cdots \wedge x_n \neq y$

each clause: 32 bits



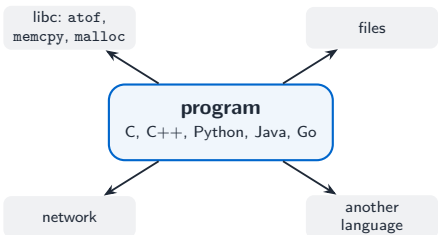
$32 \times n$

the formula grows with the path

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Challenges (5/5)

Complex and incomplete implementations



every language feature and every environment call needs a **hand-written model**

Expensive constraint solving

```
float x, y;
while (x != y)
  x = next(x);
```

floats in a loop

each iteration adds one clause

$PC: \; x_1 \neq y \wedge \cdots \wedge x_n \neq y$

each clause: 32 bits

$32 \times n$

the formula grows with the path

floats, strings: **slow** even when small

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Can an LLM Agent Reason Symbolically? (1/3)

An **LLM agent**: an LLM in a loop that calls tools and reads their results

Why it might work

- reads many languages
- knows libraries: `atof`, `memcpy`, `malloc`
- can state a constraint at a high level
- can call a solver or run code

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Can an LLM Agent Reason Symbolically? (2/3)

An **LLM agent**: an LLM in a loop that calls tools and reads their results

Why it might work

- reads many languages
- knows libraries: `atof`, `memcpy`, `malloc`
- can state a constraint at a high level
- can call a solver or run code

Why it might not

- no guarantee that an answer is correct
- seconds per query, while the solver is already the bottleneck

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Can an LLM Agent Reason Symbolically? (3/3)

An **LLM agent**: an LLM in a loop that calls tools and reads their results

Why it might work

- reads many languages
- knows libraries: `atof`, `memcpy`, `malloc`
- can state a constraint at a high level
- can call a solver or run code

Why it might not

- no guarantee that an answer is correct
- seconds per query, while the solver is already the bottleneck

Keep concolic execution, and **check every answer by running the program**

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (1/4)

```c
#define FLOATS_BETWEEN_BUG 20
int cnt = 0;

int count(float start, float end) {
  for (float cur = start; cur != end; cnt++) {
    unsigned temp;
    memcpy(&temp, &cur, sizeof(float));
    temp++;
    memcpy(&cur, &temp, sizeof(float));
  }
  if (cnt <= FLOATS_BETWEEN_BUG) {
    printf("BUG triggered!");
    return -1;
  }
}

int main(int argc, char **argv) {
  float start = atof(argv[1]), end = atof(argv[2]);
  __assume__(start < end);
  count(start, end);
}
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (2/4)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

← two command-line strings, converted to 32-bit floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (3/4)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

← bits of cur plus 1:
cur becomes the *next* float

← two command-line strings,
converted to 32-bit floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (4/4)

```c
#define FLOATS_BETWEEN_BUG 20
int cnt = 0;

int count(float start, float end) {
  for (float cur = start; cur != end; cnt++) {
    unsigned temp;
    memcpy(&temp, &cur, sizeof(float));
    temp++;
    memcpy(&cur, &temp, sizeof(float));
  }
  if (cnt <= FLOATS_BETWEEN_BUG) {
    printf("BUG triggered!");
    return -1;
  }
}

int main(int argc, char **argv) {
  float start = atof(argv[1]), end = atof(argv[2]);
  __assume__(start < end);
  count(start, end);
}
```

← bits of cur plus 1: cur becomes the *next* float

← the bug: at most 20 floats between start and end

← two command-line strings, converted to 32-bit floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (1/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 1: `./count 1.00 1.00001`

start = 1.0 0x3F800000
←end = 1.00001 0x3F800054
difference: 0x54 = 84 floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (2/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 1: `./count 1.00 1.00001`

←iteration 1: cur = 1.0

←temp = bits of cur = 0x3F800000
←temp = 0x3F800001
←cur = 1.00000012: the *next* float

start = 1.0 0x3F800000
←end = 1.00001 0x3F800054
difference: 0x54 = 84 floats

Motivating Example (3/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 1: `./count 1.00 1.00001`

←loop runs 84 times: cnt = 84

iteration	cur after it	bits of cur	cnt
1	1.00000012	0x3F800001	1
2	1.00000024	0x3F800002	2
3	1.00000036	0x3F800003	3
4	1.00000048	0x3F800004	4
5	1.0000006	0x3F800005	5
⋮	⋮	⋮	⋮
84	**1.00001001**	0x3F800054	**84**

now cur == end: the loop stops

start = 1.0 0x3F800000

←end = 1.00001 0x3F800054

difference: 0x54 = 84 floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (4/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 1: `./count 1.00 1.00001`

←loop runs 84 times: `cnt` $= 84$

←$84 \leq 20$ is false: **no bug**

`start` $= 1.0$ 0x3F800000
←`end` $= 1.00001$ 0x3F800054
difference: 0x54 $= 84$ floats

Motivating Example (5/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 2: `./count 1.0 1.000001`

start = 1.0 0x3F800000
←end = 1.000001 0x3F800008
difference: 0x08 = 8 floats

Motivating Example (6/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 2: `./count 1.0 1.000001`

←loop runs 8 times: cnt = 8

iteration	cur after it	bits of cur	cnt
1	1.00000012	0x3F800001	1
2	1.00000024	0x3F800002	2
3	1.00000036	0x3F800003	3
4	1.00000048	0x3F800004	4
5	1.0000006	0x3F800005	5
⋮	⋮	⋮	⋮
8	**1.00000095**	0x3F800008	**8**

now cur == end: the loop stops

start = 1.0 0x3F800000
←end = 1.000001 0x3F800008
difference: 0x08 = 8 floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Motivating Example (7/7)

```
1  #define FLOATS_BETWEEN_BUG 20
2  int cnt = 0;

4  int count(float start, float end) {
5    for (float cur = start; cur != end; cnt++) {
6      unsigned temp;
7      memcpy(&temp, &cur, sizeof(float));
8      temp++;
9      memcpy(&cur, &temp, sizeof(float));
10   }
11   if (cnt <= FLOATS_BETWEEN_BUG) {
12     printf("BUG triggered!");
13     return -1;
14   }
15 }

17 int main(int argc, char **argv) {
18   float start = atof(argv[1]), end = atof(argv[2]);
19   __assume__(start < end);
20   count(start, end);
21 }
```

Run 2: `./count 1.0 1.000001`

←loop runs 8 times: cnt = 8

←$8 \leq 20$ is true
←prints **BUG triggered!**

start = 1.0 0x3F800000
←end = 1.000001 0x3F800008
difference: 0x08 = 8 floats

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Conventional Concolic Execution on It $(1/4)$

Input *I*: `I[0] = "1.00"`, `I[1] = "1.00001"`: 84 iterations, line 11 is false

Conventional Concolic Execution on It (2/4)

Input I: `I[0] = "1.00"`, `I[1] = "1.00001"`: 84 iterations, line 11 is false

Symbolic store after i iterations

$$\sigma_i = \underbrace{\{\texttt{start} \mapsto \alpha,\ \texttt{end} \mapsto \beta\}}_{\text{initial state}} \cup \underbrace{\{\texttt{cnt} \mapsto i,\ \texttt{cur} \mapsto f^i(\alpha)\}}_{\text{after loop iteration } i}$$

Conventional Concolic Execution on It (3/4)

Input I: `I[0] = "1.00"`, `I[1] = "1.00001"`: 84 iterations, line 11 is false

Symbolic store after i iterations

$$\sigma_i = \underbrace{\{\texttt{start} \mapsto \alpha,\ \texttt{end} \mapsto \beta\}}_{\text{initial state}} \cup \underbrace{\{\texttt{cnt} \mapsto i,\ \texttt{cur} \mapsto f^i(\alpha)\}}_{\text{after loop iteration } i}$$

Path constraints of this run, with $n = 84$

$$PC:\ \alpha = atof(I[0]) \wedge \beta = atof(I[1]) \wedge \bigwedge_{i=0}^{n-1} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n > 20$$

Conventional Concolic Execution on It (4/4)

Input I: `I[0] = "1.00"`, `I[1] = "1.00001"`: 84 iterations, line 11 is false

Symbolic store after i iterations

$$\sigma_i = \underbrace{\{\texttt{start} \mapsto \alpha,\ \texttt{end} \mapsto \beta\}}_{\text{initial state}} \cup \underbrace{\{\texttt{cnt} \mapsto i,\ \texttt{cur} \mapsto f^i(\alpha)\}}_{\text{after loop iteration } i}$$

Path constraints of this run, with $n = 84$

$$PC:\ \alpha = atof(I[0]) \wedge \beta = atof(I[1]) \wedge \bigwedge_{i=0}^{n-1} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n > 20$$

Flip the last clause: $n \leq 20$. Now the solver must find α and β.

Why the Conventional Engine Fails (1/3)

- **External functions**: `atof` and `memcpy` are not in the source
  - add their code (large, low-level), model them by hand (costly), or
  - concretize their arguments: here the formula becomes **unsolvable**

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Why the Conventional Engine Fails (2/3)

- **External functions**: `atof` and `memcpy` are not in the source
  - add their code (large, low-level), model them by hand (costly), or
  - concretize their arguments: here the formula becomes **unsolvable**
- **Verbose formulas**: one clause per iteration, larger than the program
  - each float clause is 32 bit-level constraints

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Why the Conventional Engine Fails (3/3)

- **External functions**: `atof` and `memcpy` are not in the source
  - add their code (large, low-level), model them by hand (costly), or
  - concretize their arguments: here the formula becomes **unsolvable**
- **Verbose formulas**: one clause per iteration, larger than the program
  - each float clause is 32 bit-level constraints
- **Theory support**: KLEE has no adequate floating-point support
  - KLEE-Float finds this bug, after 10 months of extra work
  - and still fails for higher-precision numbers

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Constraints at a Higher Level (1/4)

The same path condition, written at two levels

$\alpha = \mathit{atof}(I[0]) \wedge \beta = \mathit{atof}(I[1]) \wedge$
$\bigwedge_{i<n} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n \leq 20$
(f: `memcpy` as a cast, `temp++`, `memcpy` back; floats as bit vectors)

`implementation level`

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Constraints at a Higher Level (2/4)

The same path condition, written at two levels

$\alpha = atof(I[0]) \wedge \beta = atof(I[1]) \wedge$
$\bigwedge_{i<n} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n \leq 20$
(f: `memcpy` as a cast, `temp++`, `memcpy` back; floats as bit vectors)

implementation level

LLM

Find two floating-point numbers with at most 20 representable numbers between them.

natural language (NL)

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Constraints at a Higher Level (3/4)

The same path condition, written at two levels

$\alpha = atof(I[0]) \wedge \beta = atof(I[1]) \wedge$
$\bigwedge_{i<n} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n \leq 20$
(f: `memcpy` as a cast, `temp++`, `memcpy` back; floats as bit vectors)

implementation level

LLM

Find two floating-point numbers with at most 20 representable numbers between them.

natural language (NL)

- the constraint may be NL, program code (PL), or SMT

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Constraints at a Higher Level (4/4)

The same path condition, written at two levels

$\alpha = atof(I[0]) \wedge \beta = atof(I[1]) \wedge$
$\bigwedge_{i<n} f^i(\alpha) \neq \beta \wedge f^n(\alpha) = \beta \wedge n \leq 20$
(f: memcpy as a cast, temp++, memcpy back; floats as bit vectors)

implementation level

LLM

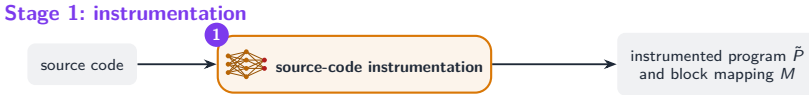
Find two floating-point numbers with at most 20 representable numbers between them.

natural language (NL)

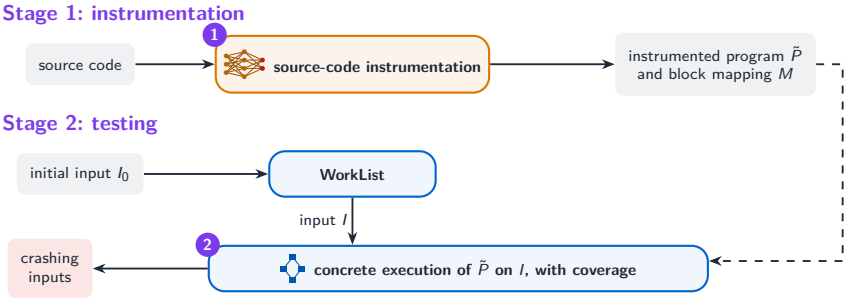
- the constraint may be NL, program code (PL), or SMT
- a **solving agent** reads it, splits the task, and calls tools (Python, Z3)

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

ConcoLLMic: Workflow (1/5)

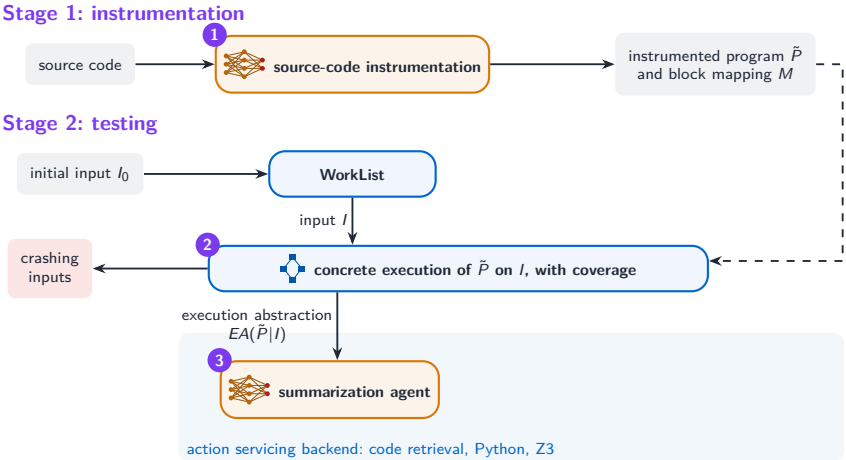


ConcoLLMic: Workflow (2/5)



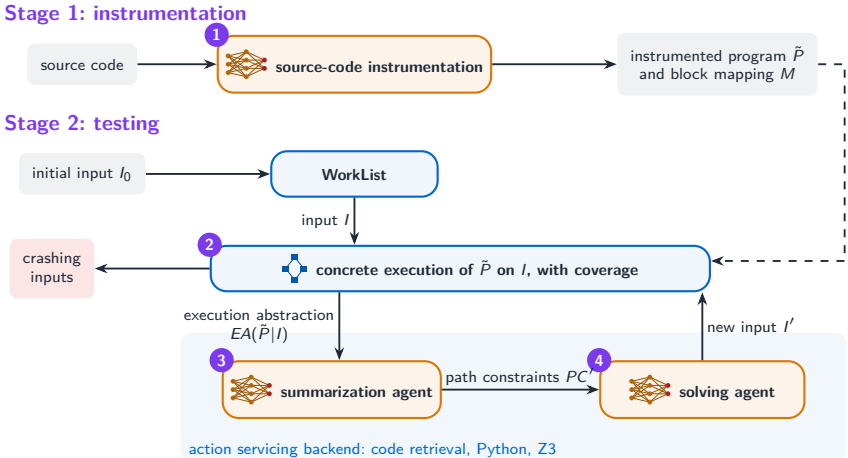
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

ConcoLLMic: Workflow (3/5)



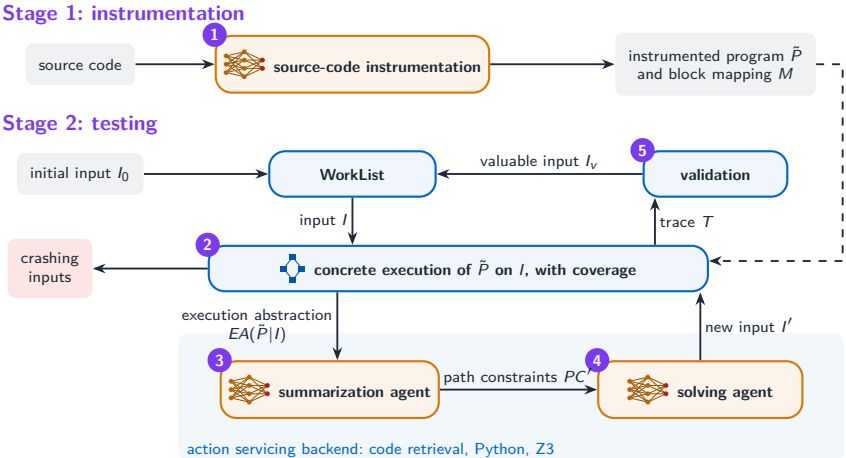
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

ConcoLLMic: Workflow (4/5)



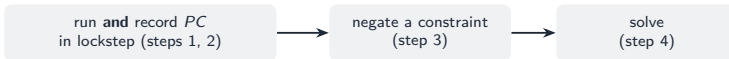
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

ConcoLLMic: Workflow (5/5)



Two Differences from Concolic Execution (1/4)

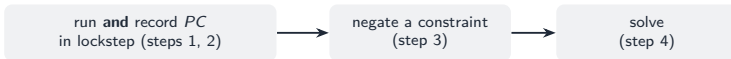
Conventional



Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Differences from Concolic Execution (2/4)

Conventional

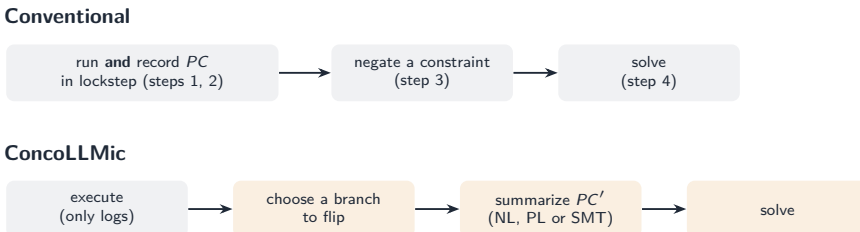


ConcoLLMic



Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

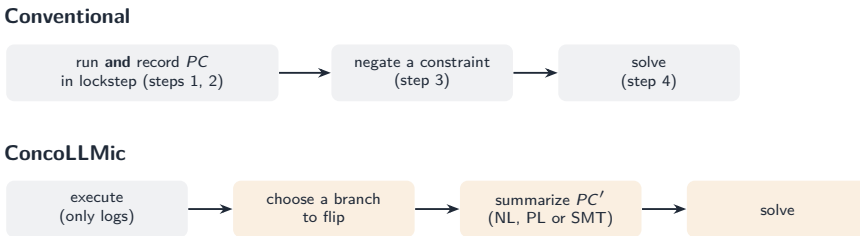
Two Differences from Concolic Execution (3/4)



- symbolize **after** the run, on a text trace: independent of the language

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Two Differences from Concolic Execution (4/4)



- symbolize **after** the run, on a text trace: independent of the language
- choose the branch **before** symbolizing: a sentence has no `for` or `if` to flip

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Stage 1: Flow-Tracing Statements (1/3)

```
 4 int count(float start, float end) {
 5  for (float cur = start; cur != end; cnt++) {
 6   fprintf(stderr, "[src/count.c] enter count 1\n");
 7   unsigned temp;
 8   memcpy(&temp, &cur, sizeof(float));
 9   temp++;
10   memcpy(&cur, &temp, sizeof(float));
11   // fprintf(stderr, "[src/count.c] exit count 1\n");
12  }

14  if (cnt <= FLOATS_BETWEEN_BUG) {
15   fprintf(stderr, "[src/count.c] enter count 2\n");
16   printf("BUG triggered!\n");
17   return -1;
18   // fprintf(stderr, "[src/count.c] exit count 2\n");
19  }
20 }

22 int main(int argc, char **argv) {
23  fprintf(stderr, "[src/count.c] enter main 1\n");
24  float start = atof(argv[1]), end = atof(argv[2]);
25  __assume__(start < end);
26  count(start, end);
27  // fprintf(stderr, "[src/count.c] exit main 1\n");
28 }
```

The pattern

```
enter func_name block_no
original code
// exit func_name block_no
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Stage 1: Flow-Tracing Statements (2/3)

```
 4 int count(float start, float end) {
 5  for (float cur = start; cur != end; cnt++) {
 6   fprintf(stderr, "[src/count.c] enter count 1\n");
 7   unsigned temp;
 8   memcpy(&temp, &cur, sizeof(float));
 9   temp++;
10   memcpy(&cur, &temp, sizeof(float));
11   // fprintf(stderr, "[src/count.c] exit count 1\n");
12  }

14  if (cnt <= FLOATS_BETWEEN_BUG) {
15   fprintf(stderr, "[src/count.c] enter count 2\n");
16   printf("BUG triggered!\n");
17   return -1;
18   // fprintf(stderr, "[src/count.c] exit count 2\n");
19  }
20 }

22 int main(int argc, char **argv) {
23  fprintf(stderr, "[src/count.c] enter main 1\n");
24  float start = atof(argv[1]), end = atof(argv[2]);
25  __assume__(start < end);
26  count(start, end);
27  // fprintf(stderr, "[src/count.c] exit main 1\n");
28 }
```

The pattern

```
enter func_name block_no
original code
// exit func_name block_no
```

the exit line is a comment: only a marker

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Stage 1: Flow-Tracing Statements (3/3)

```
 4 int count(float start, float end) {
 5  for (float cur = start; cur != end; cnt++) {
 6   fprintf(stderr, "[src/count.c] enter count 1\n");
 7   unsigned temp;
 8   memcpy(&temp, &cur, sizeof(float));
 9   temp++;
10   memcpy(&cur, &temp, sizeof(float));
11   // fprintf(stderr, "[src/count.c] exit count 1\n");
12  }

14  if (cnt <= FLOATS_BETWEEN_BUG) {
15   fprintf(stderr, "[src/count.c] enter count 2\n");
16   printf("BUG triggered!\n");
17   return -1;
18   // fprintf(stderr, "[src/count.c] exit count 2\n");
19  }
20 }

22 int main(int argc, char **argv) {
23  fprintf(stderr, "[src/count.c] enter main 1\n");
24  float start = atof(argv[1]), end = atof(argv[2]);
25  __assume__(start < end);
26  count(start, end);
27  // fprintf(stderr, "[src/count.c] exit main 1\n");
28 }
```

The pattern

```
enter func_name block_no
original code
// exit func_name block_no
```

the `exit` line is a comment: only a marker

Block mapping M

block	source lines
(main, 1)	src/count.c: 24–26
(count, 1)	src/count.c: 7–10
(count, 2)	src/count.c: 16–17

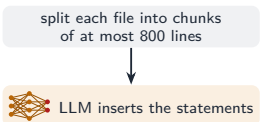
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Instrumenting with an LLM (1/4)

split each file into chunks
of at most 800 lines

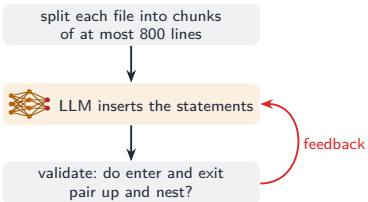
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 2)

Instrumenting with an LLM (2/4)



Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 2)

Instrumenting with an LLM (3/4)



Like matching parentheses

```
enter count 1
  ...
// exit count 1
enter count 2
  ...
// exit count 2
```

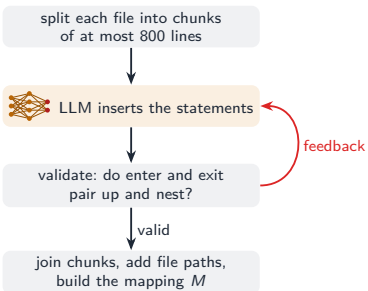
valid

```
enter count 1
  ...
enter count 2
  ...
// exit count 1
```

invalid: count 2 is not closed

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 2)

Instrumenting with an LLM (4/4)



Like matching parentheses

```
enter count 1
  ...
// exit count 1
enter count 2
  ...
// exit count 2
```

valid

```
enter count 1
  ...
enter count 2
  ...
// exit count 1
```

invalid: count 2 is not closed

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 2)

From Trace to Execution Abstraction (1/7)

Trace T of run 1, on stderr

```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (2/7)

Trace T of run 1, on `stderr`

```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (3/7)

Trace T of run 1, on `stderr`

```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

conditions and declarations stay visible; only unexecuted blocks go

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (4/7)

Trace T of run 1, on `stderr`

```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

conditions and declarations stay visible; only unexecuted blocks go

1 log has no `enter count 2`: lines 16–17 uncovered **= the target**

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (5/7)

Trace T of run 1, on `stderr`

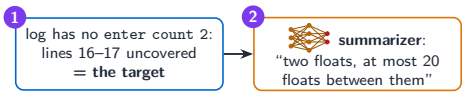
```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines enter count 1, no enter count 2

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

conditions and declarations stay visible; only unexecuted blocks go



Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (6/7)

Trace T of run 1, on `stderr`

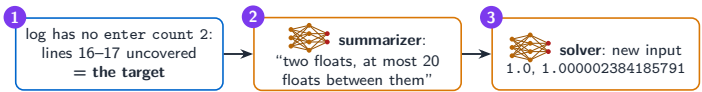
```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

conditions and declarations stay visible; only unexecuted blocks go



Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

From Trace to Execution Abstraction (7/7)

Trace T of run 1, on `stderr`

```
$ ./count 1.00 1.00001
[src/count.c] enter main 1
[src/count.c] enter count 1
[src/count.c] enter count 1
...
```

84 lines `enter count 1`, no `enter count 2`

Execution abstraction $EA(\tilde{P}|I)$

```
(1) Function call chain:
  [src/count.c] main → [src/count.c] count
(2) Executed files, unexecuted blocks deleted:
// src/count.c (28 lines total)
...
for (float cur = start; cur != end; cnt++) { ... }
if (cnt <= FLOATS_BETWEEN_BUG) {
  // Unexecuted lines 16-17 removed. Line cov: 0/2
}
...
```

conditions and declarations stay visible; only unexecuted blocks go



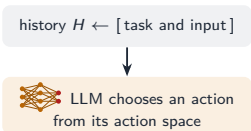
Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Stage 2: Agents in a Loop (1/4)

history $H \leftarrow$ [task and input]

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 3)

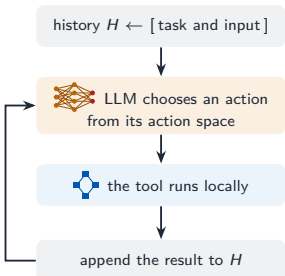
Stage 2: Agents in a Loop (2/4)



- the LLM plans with `THINK` actions

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 3)

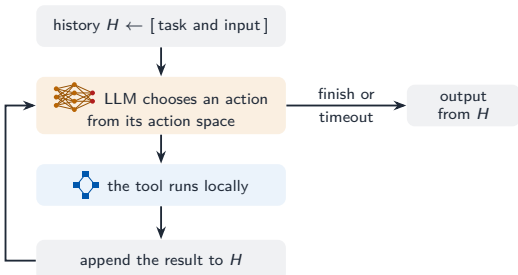
Stage 2: Agents in a Loop (3/4)



- the LLM plans with `THINK` actions
- tools give **grounded feedback**: real code, real output

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 3)

Stage 2: Agents in a Loop (4/4)



- the LLM plans with `THINK` actions
- tools give **grounded feedback**: real code, real output
- both agents use this loop, with different actions

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Algorithm 3)

The Two Action Spaces (1/3)

Summarization agent

action	what it does
`THINK`	record reasoning and plans
`CODEREQUEST`	show lines of a file, with coverage
`CHOOSEBRANCH`	pick a branch to flip, and the lines it should cover
`SUMMARIZE`	write the path constraint (NL, PL or SMT)
`FINISH`	stop for this input

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Tables 2 and 3)

The Two Action Spaces (2/3)

Summarization agent

action	what it does
`THINK`	record reasoning and plans
`CODEREQUEST`	show lines of a file, with coverage
`CHOOSEBRANCH`	pick a branch to flip, and the lines it should cover
`SUMMARIZE`	write the path constraint (NL, PL or SMT)
`FINISH`	stop for this input

Solving agent

action	what it does
`THINK`	record reasoning and plans
`EXECUTECODE`	run Python code, read its output
`QUERYSMT`	solve SMT formulas with Z3
`GENERATETEST`	write the new input as a Python harness

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Tables 2 and 3)

The Two Action Spaces (3/3)

Summarization agent

action	what it does
`THINK`	record reasoning and plans
`CODEREQUEST`	show lines of a file, with coverage
`CHOOSEBRANCH`	pick a branch to flip, and the lines it should cover
`SUMMARIZE`	write the path constraint (NL, PL or SMT)
`FINISH`	stop for this input

Solving agent

action	what it does
`THINK`	record reasoning and plans
`EXECUTECODE`	run Python code, read its output
`QUERYSMT`	solve SMT formulas with Z3
`GENERATETEST`	write the new input as a Python harness

Output of summarization: triples (`target_branch`, `path_constraint`, `lines_to_cover`)

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Tables 2 and 3)

Summarization Agent on the Example (1/5)

Input: I and its execution abstraction; tools: the five actions

`THINK` Let me list and analyse all unexecuted branches ...

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Summarization Agent on the Example (2/5)

Input: I and its execution abstraction; tools: the five actions

THINK

Let me list and analyse all unexecuted branches ...

CODEREQUEST

file: src/count.c, line_range: 16-17

```
|16|Uncovered|: printf("BUG triggered!");  |17|Uncovered|: return -1;
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Summarization Agent on the Example (3/5)

Input: I and its execution abstraction; tools: the five actions

THINK

Let me list and analyse all unexecuted branches . . .

CODEREQUEST

file: `src/count.c`, line_range: 16-17

```
|16|Uncovered|: printf("BUG triggered!");  |17|Uncovered|: return -1;
```

CHOOSEBRANCH

target_branch: `if (cnt <= FLOATS_BETWEEN_BUG) => true`
rationale: a key program behavior; its block has 0% line coverage
lines_to_cover: `src/count.c:16-17`

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Summarization Agent on the Example (4/5)

Input: I and its execution abstraction; tools: the five actions

THINK

Let me list and analyse all unexecuted branches . . .

CODEREQUEST

file: `src/count.c`, `line_range: 16-17`

```
|16|Uncovered|: printf("BUG triggered!");  |17|Uncovered|: return -1;
```

CHOOSEBRANCH

`target_branch: if (cnt <= FLOATS_BETWEEN_BUG) => true`
rationale: a key program behavior; its block has 0% line coverage
`lines_to_cover: src/count.c:16-17`

SUMMARIZE

path_constraint:
a) The program should take 2 CLI arguments `start` and `end`.
b) They are both valid FP numbers with `start` $<$ `end`.
c) The number of distinct IEEE-754 single-precision values in [`start`, `end`) must be ≤ 20.

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Summarization Agent on the Example (5/5)

Input: *I* and its execution abstraction; tools: the five actions

THINK	Let me list and analyse all unexecuted branches . . .
CODEREQUEST	file: `src/count.c`, line_range: 16–17
	`|16|Uncovered|: printf("BUG triggered!"); |17|Uncovered|: return -1;`
CHOOSEBRANCH	target_branch: `if (cnt <= FLOATS_BETWEEN_BUG) => true` rationale: a key program behavior; its block has 0% line coverage lines_to_cover: `src/count.c:16-17`
SUMMARIZE	path_constraint: a) The program should take 2 CLI arguments `start` and `end`. b) They are both valid FP numbers with `start` $<$ `end`. c) The number of distinct IEEE-754 single-precision values in [`start`, `end`) must be ≤ 20.
FINISH	task_completed: True

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Solving Agent on the Example (1/4)

Input: the constraints a), b), c) and the old input I; tools: the four actions

`THINK`

The original input [1.0, 1.00001) does not meet the constraints: it has over 20 floats.
Let me calculate a suitable `end` if `start` is 1.0.

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Solving Agent on the Example (2/4)

Input: the constraints a), b), c) and the old input I; tools: the four actions

`THINK`

The original input [1.0, 1.00001) does not meet the constraints: it has over 20 floats.
Let me calculate a suitable `end` if `start` is 1.0.

`THINK`

Near 1.0 the smallest increment is about $2^{-23} \approx 1.19 \times 10^{-7}$. For at most 20 values,
the range is about $20 \times 1.19 \times 10^{-7} \approx 2.38 \times 10^{-6}$. Let me verify how floats are distributed.

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Solving Agent on the Example (3/4)

Input: the constraints a), b), c) and the old input I; tools: the four actions

THINK

The original input [1.0, 1.00001) does not meet the constraints: it has over 20 floats.
Let me calculate a suitable end if start is 1.0.

THINK

Near 1.0 the smallest increment is about $2^{-23} \approx 1.19 \times 10^{-7}$. For at most 20 values,
the range is about $20 \times 1.19 \times 10^{-7} \approx 2.38 \times 10^{-6}$. Let me verify how floats are distributed.

EXECUTECODE

```
def next_float(f): ... # calculate the next float
current = 1.0
for i in range(19): current = next_float(current)
print(next_float(current))  # the 20th value
```

```
stdout = 1.000002384185791
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Solving Agent on the Example (4/4)

Input: the constraints a), b), c) and the old input I; tools: the four actions

THINK

The original input [1.0, 1.00001) does not meet the constraints: it has over 20 floats.
Let me calculate a suitable `end` if `start` is 1.0.

THINK

Near 1.0 the smallest increment is about $2^{-23} \approx 1.19 \times 10^{-7}$. For at most 20 values, the range is about $20 \times 1.19 \times 10^{-7} \approx 2.38 \times 10^{-6}$. Let me verify how floats are distributed.

EXECUTECODE

```
def next_float(f): ... # calculate the next float
current = 1.0
for i in range(19): current = next_float(current)
print(next_float(current))  # the 20th value
```

```
stdout = 1.000002384185791
```

GENERATETEST

is_satisfiable: True
exec_program: `subprocess.run('count 1.0 1.000002384185791') ...`

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Case Study: A Failing malloc (1/4)

Target in bc, the GNU calculator

```
1 void *bc_malloc(size_t size) {
2   void *ptr = (void *) malloc(size);
3   if (ptr == NULL) {
4     // Unexecuted line 723 removed.
5   }
6   return ptr;
7 }
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Case Study: A Failing malloc (2/4)

Target in bc, the GNU calculator

```
1 void *bc_malloc(size_t size) {
2   void *ptr = (void *) malloc(size);
3   if (ptr == NULL) {
4     // Unexecuted line 723 removed.
5   }
6   return ptr;
7 }
```

Path constraints

- the program calls bc_malloc
- malloc(size) returns NULL

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Case Study: A Failing malloc (3/4)

Target in bc, the GNU calculator

```
1 void *bc_malloc(size_t size) {
2   void *ptr = (void *) malloc(size);
3   if (ptr == NULL) {
4     // Unexecuted line 723 removed.
5   }
6   return ptr;
7 }
```

Path constraints

- the program calls `bc_malloc`
- `malloc(size)` returns `NULL`

The solving agent's harness (excerpt)

```
void *malloc(size_t size) { // malloc_wrapper.c
  if (real_malloc == NULL)
    real_malloc = dlsym(RTLD_NEXT, "malloc");
  if (++malloc_count >= fail_after) // 30
    return NULL;
  return real_malloc(size);
}
```

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

Case Study: A Failing malloc (4/4)

Target in bc, the GNU calculator

```
1 void *bc_malloc(size_t size) {
2   void *ptr = (void *) malloc(size);
3   if (ptr == NULL) {
4     // Unexecuted line 723 removed.
5   }
6   return ptr;
7 }
```

Path constraints

- the program calls `bc_malloc`
- `malloc(size)` returns `NULL`

The solving agent's harness (excerpt)

```
void *malloc(size_t size) { // malloc_wrapper.c
 if (real_malloc == NULL)
   real_malloc = dlsym(RTLD_NEXT, "malloc");
 if (++malloc_count >= fail_after) // 30
  return NULL;
 return real_malloc(size);
}
```

```
gcc -shared -fPIC malloc_wrapper.c -o malloc_wrapper.so
-ldl
env["LD_PRELOAD[1]"] = "./malloc_wrapper.so"
subprocess.run(["./bc/bc", "non_existent_file.bc"],
env=env)
```

harness.py

[1] `LD_PRELOAD`: on Linux, a shared library listed here is loaded before all others, so its `malloc` replaces the one in libc.

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026

ConcoLLMic: The Tool (1/5)

ConcoLLMic

Luo, Zhao, Wolff, Cadar, Roychoudhury
NUS and Imperial College London
IEEE S&P 2026

GitHub:
ConcoLLMic/ConcoLLMic

KLEE took years to build, and supports C only. Can LLM agents be the symbolic engine instead?

ConcoLLMic: The Tool (2/5)

ConcoLLMic

Luo, Zhao, Wolff, Cadar, Roychoudhury
NUS and Imperial College London
IEEE S&P 2026

GitHub:
ConcoLLMic/ConcoLLMic

KLEE took years to build, and supports C only. Can LLM agents be the symbolic engine instead?

- 8.2k lines of Python; Claude 3.7 Sonnet in the paper

ConcoLLMic: The Tool (3/5)

ConcoLLMic

Luo, Zhao, Wolff, Cadar, Roychoudhury
NUS and Imperial College London
IEEE S&P 2026

GitHub:
ConcoLLMic/ConcoLLMic

KLEE took years to build, and supports C only. Can LLM agents be the symbolic engine instead?

- 8.2k lines of Python; Claude 3.7 Sonnet in the paper
- temperature 0 for instrumentation, 0.5 for the two agents

ConcoLLMic: The Tool (4/5)

ConcoLLMic

Luo, Zhao, Wolff, Cadar, Roychoudhury
NUS and Imperial College London
IEEE S&P 2026

GitHub:
ConcoLLMic/ConcoLLMic

KLEE took years to build, and supports C only. Can LLM agents be the symbolic engine instead?

- 8.2k lines of Python; Claude 3.7 Sonnet in the paper
- temperature 0 for instrumentation, 0.5 for the two agents
- prompt caching; one solving pipeline per constraint, in parallel

ConcoLLMic: The Tool (5/5)

ConcoLLMic

Luo, Zhao, Wolff, Cadar, Roychoudhury
NUS and Imperial College London
IEEE S&P 2026

GitHub:
ConcoLLMic/ConcoLLMic

KLEE took years to build, and supports C only. Can LLM agents be the symbolic engine instead?

- 8.2k lines of Python; Claude 3.7 Sonnet in the paper
- temperature 0 for instrumentation, 0.5 for the two agents
- prompt caching; one solving pipeline per constraint, in parallel

November 2026: 165 stars, 18 commits, no release; GPLv3 for non-commercial use; the README now defaults to Claude 4.5 Sonnet

Evaluation Setup (1/3)

subject	language	kLoC	input
woff2	C++	56	font data
oggenc	C	83	audio, CLI
bc	C	29	math script, CLI
libmatheval	C	15	math expression
libyaml	C	11	YAML file
libsoup	C	71	HTTP message
krep*	C	4	regex, CLI
confetti*	C	9	config file
ultrajson	Python, C	9	JSON, CLI
jansi	Java, C	72	ANSI codes, CLI
py4j	Python, Java	21	program, object
protobuf-go	Go, C++	36	schema, CLI

* created in 2025, after the model's training cut-off

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Table 4, Section 4.1)

Evaluation Setup (2/3)

subject	language	kLoC	input
woff2	C++	56	font data
oggenc	C	83	audio, CLI
bc	C	29	math script, CLI
libmatheval	C	15	math expression
libyaml	C	11	YAML file
libsoup	C	71	HTTP message
krep*	C	4	regex, CLI
confetti*	C	9	config file
ultrajson	Python, C	9	JSON, CLI
jansi	Java, C	72	ANSI codes, CLI
py4j	Python, Java	21	program, object
protobuf-go	Go, C++	36	schema, CLI

* created in 2025, after the model's training cut-off

Compared with

- KLEE, KLEE-Float, KLEE-Pending
- SymCC, SymSan (compiled concolic)
- AFL++ (greybox fuzzer)

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Table 4, Section 4.1)

Evaluation Setup (3/3)

subject	language	kLoC	input
woff2	C++	56	font data
oggenc	C	83	audio, CLI
bc	C	29	math script, CLI
libmatheval	C	15	math expression
libyaml	C	11	YAML file
libsoup	C	71	HTTP message
krep*	C	4	regex, CLI
confetti*	C	9	config file
ultrajson	Python, C	9	JSON, CLI
jansi	Java, C	72	ANSI codes, CLI
py4j	Python, Java	21	program, object
protobuf-go	Go, C++	36	schema, CLI

* created in 2025, after the model's training cut-off

Compared with

- KLEE, KLEE-Float, KLEE-Pending
- SymCC, SymSan (compiled concolic)
- AFL++ (greybox fuzzer)

Setup

- 48 h per run, 5 runs, 2 CPUs, 8 GiB
- ConcoLLMic stops after 30 min with no new coverage
- plus FP-Bench: 26 floating-point programs

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Table 4, Section 4.1)

Eleven New Bugs

#	subject	bug	status
1	oggenc	null pointer dereference, audio file with invalid format	fixed
2	oggenc	signed integer overflow when resampling	reported
3	oggenc	signed integer overflow, WAV file with malformed INFO chunk size	fixed
4	oggenc	memory leak in `wav_open()`, invalid WAV header	fixed
5	krep	wrong handling of arguments in string mode `-s`	fixed
6	libyaml	memory leak, invalid UTF-8 input	fixed
7	libyaml	memory leak, incomplete UTF-8 octet sequence	confirmed
8	libsoup	integer overflow in `soup_cookie_parse()`, malformed “expired” value	fixed
9	confetti	memory leak after a partially malformed config input	fixed
10	ultrajson	wrong exception handling, JSON string with nested keys	fixed
11	jansi	`ClassCastException`, ANSI escape codes with quoted arguments	reported

Luo, Zhao, Wolff, Cadar, Roychoudhury, IEEE S&P 2026 (Section 4.3, Table 6)